

Java Collection Framework

Ultimate Interview Preparation Guide

In-Depth Internals | Diagrams | Code Examples | Interview Q&A

LIST

SET

MAP

QUEUE

Table of Contents

Table of Contents.....	2
0. What is the Collections Framework?	3

0. What is the Collections Framework?

The Java Collections Framework (JCF) is a unified architecture for representing and manipulating groups of objects. It was introduced in Java 1.2 and lives in the `java.util` package.

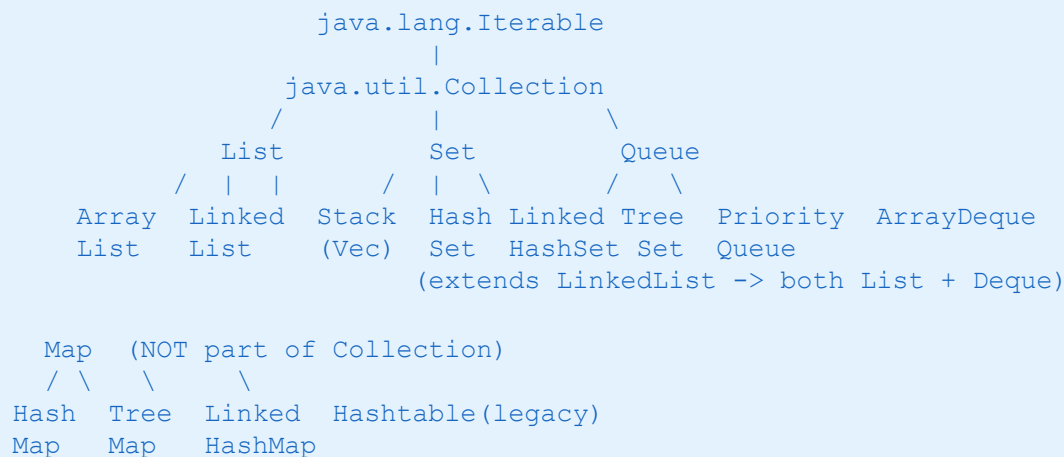
Why was it introduced?

- Before JCF, Java had Arrays, Vector, Hashtable — inconsistent APIs
- JCF provides a standard, reusable set of data structures and algorithms
- Reduces programming effort, increases performance, encourages software reuse

Three pillars of JCF

- Interfaces — Abstract data types (List, Set, Map, Queue, Deque)
- Implementations — Concrete classes (ArrayList, HashMap, TreeSet...)
- Algorithms — Static methods in `java.util.Collections` (sort, shuffle, binarySearch...)

Complete Hierarchy Diagram



Key Interfaces Summary

Interface	Package	Extends	Key Feature
Collection	<code>java.util</code>	Iterable	Root of collection hierarchy
List	<code>java.util</code>	Collection	Ordered, index-based, duplicates OK
Set	<code>java.util</code>	Collection	No duplicates
Queue	<code>java.util</code>	Collection	FIFO ordering
Deque	<code>java.util</code>	Queue	Double-ended queue
Map	<code>java.util</code>	—	Key-value pairs, NOT a Collection
SortedSet	<code>java.util</code>	Set	Elements in sorted order

Interface	Package	Extends	Key Feature
SortedMap	java.util	Map	Keys in sorted order
NavigableSet	java.util	SortedSet	Navigation methods (floor, ceiling...)
NavigableMap	java.util	SortedMap	Navigation methods

1. LIST INTERFACE

Core Characteristics

- Growable in nature — resizes automatically (no fixed size like arrays)
- Insertion order preserved — elements stay in the order they were added
- Allows DUPLICATE elements
- Index-based — every element has an index starting from 0
- Allows RANDOM ACCESS — get element at any index directly
- Allows NULL values
- Extends Collection, which extends Iterable

★ **Interview Q:** List preserves insertion order. This makes it different from Set (no order) and Map (key-based). The most common List used in interviews is ArrayList.

1.1 ArrayList — Internal Implementation

ArrayList is backed by a **dynamic Object[] array** called **elementData**.

ArrayList Internal Structure:

```

elementData (Object[]):                                size = 4
+-----+-----+-----+-----+-----+-----+-----+
| 'A' | 'B' | 'C' | 'D' | null | null | null | null |
+-----+-----+-----+-----+-----+-----+-----+
Index:    0       1       2       3       4       5       6
          ^-- actual elements ----^   ^-- empty slots (capacity) --^

capacity = 7 (array size),   size = 4 (actual element count)
  
```

- Default initial capacity: 10
- When size == capacity, it grows: newCapacity = oldCapacity + (oldCapacity >> 1) i.e. 1.5x growth
- Growth example: 10 → 15 → 22 → 33 → ...
- Uses Arrays.copyOf() to copy elements to new bigger array during resize
- Random access: O(1) — direct index lookup into array
- Add at end: O(1) amortized (occasionally O(n) for resize)
- Add/remove in middle: O(n) — requires shifting elements

ArrayList — Memory Layout During Insertion

Step 1: new ArrayList<>() add at index 1	Step 2: add('A','B','C')	Step 3:
+-----+-----+-----+-----+-----+-----+-----+ +-----+-----+-----+-----+-----+-----+-----+	+-----+-----+-----+-----+-----+-----+-----+ +-----+-----+-----+-----+-----+-----+-----+	+-----+-----+-----+-----+-----+-----+-----+ +-----+-----+-----+-----+-----+-----+-----+

capacity=5	A B C size=3	A X B
C size=4		
+---+---+---+---+	+---+---+---+---+	+---+---+
+---+---+		
size=0	index: 0 1 2	Shift:
B,C moved right		

ArrayList — Code Snippet

```
import java.util.ArrayList;
import java.util.Collections;

ArrayList<String> list = new ArrayList<>(); // capacity=10
ArrayList<String> list2 = new ArrayList<>(50); // custom capacity

list.add("A"); // add at end O(1)
list.add(1, "B"); // add at index O(n)
list.get(0); // random access O(1)
list.remove(0); // remove by idx O(n) -- shifting happens
list.remove("A"); // remove by val O(n) -- linear scan
list.size(); // size O(1)
list.contains("A"); // search O(n) -- linear scan

Collections.sort(list); // sort O(n log n)
Collections.binarySearch(list, "B"); // O(log n) - list must be sorted
```

★ **Interview Q:** Why is ArrayList not thread-safe? ArrayList uses no synchronization. Two threads adding simultaneously can corrupt the elementData array. Use Collections.synchronizedList(list) or CopyOnWriteArrayList for thread safety.

1.2 Vector — Internal Implementation

Vector is identical to ArrayList internally BUT every method is **synchronized** (thread-safe). It is a legacy class from Java 1.0.

Vector Internal Structure (same array-based as ArrayList):

synchronized get() {	synchronized add() {	synchronized
remove() {		
lock object;	lock object;	lock object;
return elementData[i];	elementData[size++] = e;	shift elements
left;		
unlock object;	unlock object;	unlock object;
}	}	}

Growth factor: doubles (x2) — newCapacity = 2 * oldCapacity
 ArrayList grows by 1.5x — Vector is more memory-wasteful

- Default capacity: 10, grows by capacityIncrement (if set) or doubles

- Every public method is synchronized — one thread at a time
- Thread-safe BUT slower than ArrayList due to lock overhead
- Prefer: Collections.synchronizedList(new ArrayList<>()) or CopyOnWriteArrayList

★ **Interview Q:** Vector vs ArrayList: (1) Vector is synchronized; ArrayList is not. (2) Vector doubles in size; ArrayList grows by 1.5x. (3) Vector is legacy (Java 1.0); ArrayList is modern (Java 1.2). Use ArrayList unless you specifically need a thread-safe legacy class.

1.3 Stack — Internal Implementation

Stack extends **Vector** and adds 5 stack-specific methods. It follows **LIFO (Last In, First Out)**.

Stack (LIFO) — TOP is the last element in the Vector array:

Operation: push('A'), push('B'), push('C')

Internal Vector array:	TOP	
+-----+-----+-----+		
'A' 'B' 'C'	v	
+-----+-----+-----+		
index: 0 1 2		['C'] <- popped first
		['B']
		['A']

pop() -> removes index 2 ('C') -> size--
 peek() -> returns elementData[size-1] without removing
 search('A') -> returns distance from top = 3

- push(E e) → calls addElement(e) — adds at top (end of array) O(1)
- pop() → calls removeElementAt(size-1) O(1)
- peek() → returns elementData[size-1] without removing O(1)
- isEmpty() → returns size == 0 O(1)
- search(obj) → returns 1-based position from top O(n)

```
Stack<Integer> stack = new Stack<>();
stack.push(10);      // [10]
stack.push(20);      // [10, 20]
stack.push(30);      // [10, 20, 30]
stack.peek();        // 30 (does NOT remove)
stack.pop();         // 30 removed -> [10, 20]
stack.search(10);     // 2 (10 is 2nd from top)

// PREFER ArrayDeque over Stack:
Deque<Integer> betterStack = new ArrayDeque<>();
betterStack.push(10);
betterStack.pop();
```

★ **Interview Q:** Why prefer ArrayDeque over Stack? Stack extends Vector which is synchronized — unnecessary overhead. ArrayDeque is faster, not legacy, and Deque interface is cleaner. Always say: 'In modern Java, use ArrayDeque as a stack.'

1.4 LinkedList — Internal Implementation

LinkedList is backed by a **doubly linked list**. Each element is stored in a Node object with prev, data, and next pointers.

LinkedList Internal Node Structure:

```
class Node<E> {
    E item;
    Node<E> prev;
    Node<E> next;
}
```

LinkedList containing ['A', 'B', 'C', 'D']:

```
first                                     last
|                                       |
v                                       v
+-----+ +-----+ +-----+ +-----+
| null |<-->| 'A' |<-->| 'B' |<-->| 'C' |<-->| 'D' | null |
| 'A' |   | 'B' |   | 'C' |   | 'D' |
+-----+ +-----+ +-----+ +-----+
HEAD                                     TAIL
```

```
addFirst(): new node -> prev=null, next=old_first  O(1)
addLast():  new node -> prev=old_last, next=null   O(1)
get(i):     traverse from head to index i         O(n)
```

- Implements List, Deque, Queue — most versatile collection
- No random access — get(i) traverses from head or tail (whichever is closer)
- Insertion/deletion at head or tail: O(1)
- Insertion/deletion in middle: O(n) to find the node, O(1) to insert
- More memory per element: each Node holds 3 references (item, prev, next)
- size field maintained separately — size() is O(1)

```
LinkedList<String> ll = new LinkedList<>();
ll.add("B");           // addLast  O(1)
ll.addFirst("A");      // O(1)   -> [A, B]
ll.addLast("C");       // O(1)   -> [A, B, C]
ll.get(1);             // O(n)   -> 'B' (traversal!)
ll.getFirst();         // O(1)   -> 'A'
ll.getLast();          // O(1)   -> 'C'
ll.removeFirst();      // O(1)   -> removes 'A'

// As a Queue:
ll.offer("X");         // enqueue at tail
ll.poll();             // dequeue from head

// As a Stack:
ll.push("X");          // push at head
ll.pop();              // pop from head
```


ArrayList vs LinkedList — Definitive Comparison

Operation	ArrayList	LinkedList	Reason
get(i)	O(1)	O(n)	Array index vs node traversal
add(e) at end	O(1)*	O(1)	*amortized; O(n) on resize
add(i, e) middle	O(n)	O(n)	ArrayList: shift; LinkedList: find node
remove(i)	O(n)	O(n)	ArrayList: shift; LinkedList: find node
removeFirst()	O(n)	O(1)	LinkedList wins — just update pointer
removeLast()	O(n)	O(1)	LinkedList wins — tail pointer
contains(e)	O(n)	O(n)	Both need linear scan
Memory usage	Less	More	LinkedList: 3 refs per node
Cache friendliness	High	Low	Array is contiguous in memory
Thread safety	No	No	Neither is synchronized

2. SET INTERFACE

Core Characteristics

- Growable in nature
- NO duplicate elements — add() returns false if element exists
- NOT index-based — no get(i) method
- Insertion order NOT guaranteed by default
- Uses hashCode() and equals() internally to determine uniqueness
- At most one null allowed (depends on implementation)

★ **Interview Q:** How does Set detect duplicates? When you call add(e): (1) compute hashCode(e) to find bucket, (2) if bucket is empty → insert, (3) if bucket has elements → call equals() on each. If any equals() returns true → duplicate found, don't insert.

2.1 HashSet — Internal Implementation

HashSet is internally backed by a **HashMap<E, Object>**. The element is stored as the KEY, and a dummy PRESENT object is the value.

```
HashSet<String> set = new HashSet<>();
set.add("cat"); set.add("dog"); set.add("bat");
```

Internally this HashMap:

Key	Value
'cat'	PRESENT
'dog'	PRESENT
'bat'	PRESENT

<- PRESENT = new Object() (dummy placeholder)

HashMap internal buckets (array of linked lists / trees):

```
bucket[0]: null
bucket[1]: --> ['cat', PRESENT] --> ['bat', PRESENT] (same hash bucket)
bucket[2]: --> ['dog', PRESENT]
bucket[3]: null ...
```

add('cat') again? hashCode('cat') -> bucket[1] -> equals('cat') -> TRUE
-> NOT added

- Default initial capacity: 16, load factor: 0.75
- Rehashing occurs when size > capacity * loadFactor
- After Java 8: buckets switch from linked list to Red-Black Tree when > 8 nodes (treeify)
- Average O(1) for add, remove, contains; worst O(n) for bad hashCode
- Allows ONE null element (null stored at bucket[0])
- NOT thread-safe — use Collections.synchronizedSet() or ConcurrentHashMap.newKeySet()

```

HashSet<String> set = new HashSet<>();
set.add("apple");    // true - added
set.add("banana");  // true - added
set.add("apple");    // false - duplicate, NOT added
set.add(null);       // true - one null allowed
set.contains("banana"); // true
set.size();          // 3

// Override hashCode + equals for custom objects:
// If two objects are equal (equals=true), they MUST have same hashCode
// Violating this breaks HashSet / HashMap behavior!

```

2.2 LinkedHashMap — Internal Implementation

LinkedHashSet extends **HashSet**. It uses a **LinkedHashMap** internally which maintains a doubly-linked list across all entries to preserve insertion order.

LinkedHashSet adds a doubly-linked list over the hash table:

Hash table buckets:	Insertion-order linked list:
[0]: 'cat'	head --> 'cat' <--> 'dog' <--> 'bat'
<-- tail	
[1]: 'dog'	(order: cat was added first, bat
last)	
[2]: 'bat'	

Iteration follows the linked list -> cat, dog, bat (insertion order)
 Hashing still used for O(1) add/remove/contains

- Maintains insertion order — elements returned in the order added
- Slightly slower than HashSet — extra pointer maintenance per operation
- Same capacity and load factor defaults as HashSet (16, 0.75)
- Allows ONE null

★ **Interview Q:** When to use **LinkedHashSet** over **HashSet**? Use **LinkedHashSet** when you need both uniqueness AND insertion order. Example: collecting unique visited pages in the order visited.

2.3 TreeSet — Internal Implementation

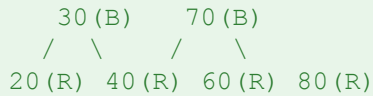
TreeSet is backed by a **TreeMap** which uses a **Red-Black Tree** — a self-balancing BST. Elements are always kept in sorted order.

TreeSet with elements: 50, 30, 70, 20, 40, 60, 80

```

      50 (BLACK root)
     /  \

```

**Red-Black Tree Properties:**

1. Every node is RED or BLACK
2. Root is always BLACK
3. No two adjacent RED nodes
4. Every path from root to null has same number of BLACK nodes
5. Guarantees $O(\log n)$ for all operations

Iteration order: 20, 30, 40, 50, 60, 70, 80 (in-order traversal = sorted)

- $O(\log n)$ for add, remove, contains — guaranteed (no worst case like HashSet)
- Does NOT allow null — `compareTo(null)` throws `NullPointerException`
- Elements must implement `Comparable` OR provide a `Comparator` at construction
- Implements `NavigableSet` — provides `floor()`, `ceiling()`, `higher()`, `lower()`, `headSet()`, `tailSet()`

```

TreeSet<Integer> ts = new TreeSet<>();
ts.add(50); ts.add(30); ts.add(70); ts.add(20);
// Automatically sorted: [20, 30, 50, 70]

ts.first();           // 20   (smallest)
ts.last();            // 70   (largest)
ts.floor(35);         // 30   (largest element <= 35)
ts.ceiling(35);       // 50   (smallest element >= 35)
ts.higher(30);        // 50   (strictly greater than 30)
ts.lower(50);         // 30   (strictly less than 50)
ts.headSet(50);       // [20, 30] (elements < 50)
ts.tailSet(50);       // [50, 70] (elements >= 50)
ts.subSet(30, 70);    // [30, 50] (30 inclusive, 70 exclusive)

// Custom ordering with Comparator:
TreeSet<String> byLength = new
TreeSet<>(Comparator.comparingInt(String::length));

```

Feature	HashSet	LinkedHashSet	TreeSet
Order	No order (random)	Insertion order	Sorted (natural/comparator)
Null allowed	Yes (1 null)	Yes (1 null)	NO (NullPointerException)
Performance	$O(1)$ avg	$O(1)$ avg	$O(\log n)$ guaranteed
Duplicates	Not allowed	Not allowed	Not allowed
Internal struct	HashMap	LinkedHashMap	TreeMap (Red-Black Tree)
Thread-safe	No	No	No

Feature	HashSet	LinkedHashSet	TreeSet
Implements	Set	Set	NavigableSet, SortedSet, Set

3. MAP INTERFACE

Core Characteristics

- Stores KEY-VALUE pairs — each pair is a `Map.Entry<K,V>` (also called Entity)
- Keys are UNIQUE — duplicate key overwrites the old value
- Values CAN be duplicated
- NOT index-based — access elements via keys
- Map does NOT extend Collection interface — separate hierarchy
- Allows null keys and values (depends on implementation)

★ **Interview Q:** Map is NOT a subtype of Collection — this is a very common trick question. Map has its own hierarchy. It does NOT extend Collection or Iterable. You cannot use an enhanced for-loop directly on a Map (use `entrySet()`, `keySet()`, or `values()`).

Map.Entry<K,V> — The Building Block

Every key-value pair in a Map is stored as a `Map.Entry` object:

```
Map.Entry<String, Integer> {
    K key    = "Alice";
    V value  = 95;
}

// How to iterate a Map using entrySet():
for (Map.Entry<String, Integer> entry : map.entrySet()) {
    System.out.println(entry.getKey() + " -> " + entry.getValue());
}
```

Key Methods

Method	Description	Return Type
<code>put(K key, V val)</code>	Add/update key-value pair	V (old value or null)
<code>get(Object key)</code>	Get value by key	V (or null if not found)
<code>getOrDefault(key, def)</code>	Get value or default if missing	V
<code>remove(Object key)</code>	Remove by key	V (removed value)
<code>containsKey(key)</code>	Check if key exists	boolean
<code>containsValue(val)</code>	Check if value exists	boolean
<code>keySet()</code>	All keys	<code>Set<K></code>
<code>values()</code>	All values	<code>Collection<V></code>
<code>entrySet()</code>	All key-value pairs	<code>Set<Map.Entry<K,V>></code>
<code>size()</code>	Number of key-value pairs	int

Method	Description	Return Type
<code>isEmpty()</code>	True if no entries	boolean
<code>putIfAbsent(k, v)</code>	Add only if key not present	V
<code>computeIfAbsent(k, fn)</code>	Add with function if absent	V
<code>merge(k, v, fn)</code>	Merge with existing value	V

3.1 HashMap — Internal Implementation

HashMap uses an array of `Node<K,V>[]` (called **table**). Each bucket is a linked list (Java 8+ becomes a Red-Black Tree when bucket has > 8 nodes).

```
HashMap<String,Integer> map - capacity=16, loadFactor=0.75

table[] (Node array, size=16):
+-----+-----+-----+-----+-----+-----+-----+-----+
| [0] | [1] | [2] | [3] | [4] | [5] | [6] | ... | [15] |
+-----+-----+-----+-----+-----+-----+-----+
|      |      |      |      |      |      |      |
v      v      v
[null] Node{      Node{
      key="Alice"  key="Bob"
      val=95       val=88
      hash=...     hash=...
      next=null    next=Node{key="Eve",val=72,next=null}}
      ^-- collision: Alice & Bob & Eve hash to same

bucket

Java 8 optimization: if bucket nodes > 8 -> convert to Red-Black Tree
This prevents O(n) worst case -> O(log n)
If nodes drop below 6 -> convert back to linked list (untreeify)
```

- Default capacity: 16, load factor: 0.75, threshold = $16 * 0.75 = 12$
- When entries > threshold: resize to double capacity and rehash all entries
- Rehashing: recompute hash for every entry and place in new table — $O(n)$
- `put()` flow: `hashCode(key)` → `hash` → `index = (n-1) & hash` → traverse bucket → insert/update
- Allows ONE null key (always stored at index 0) and multiple null values
- NOT thread-safe

```
HashMap put(key, value) flow:

put("Alice", 95)
|
v
hash = hashCode("Alice") ^ (hashCode >> 16)  <- spreads bits
|
v
index = (capacity - 1) & hash  <- maps to bucket array index
|
```

```

    v
    Is bucket empty?
    YES -> insert new Node{key,val,hash,next=null}
    NO  -> traverse linked list:
            found same key?  YES -> update value
            NOT found        -> add new Node at end of list
    |
    v
    size++; if size > threshold -> resize(capacity * 2)

```

```

HashMap<String, Integer> map = new HashMap<>();
map.put("Alice", 95);
map.put("Bob", 88);
map.put("Alice", 99); // overwrites 95 -> returns old value 95
map.get("Bob"); // 88
map.getDefault("Eve", 0); // 0 (Eve doesn't exist)
map.containsKey("Bob"); // true
map.remove("Bob"); // removes Bob entry

// Iterate:
for (Map.Entry<String,Integer> e : map.entrySet())
    System.out.println(e.getKey() + " = " + e.getValue());

// putIfAbsent - won't overwrite existing:
map.putIfAbsent("Alice", 50); // Alice stays 99

// computeIfAbsent:
map.computeIfAbsent("Charlie", k -> k.length()); // Charlie=7

```

★ **Interview Q:** What happens if two keys have the same hashCode? This is called a COLLISION. They go into the same bucket and form a linked list. Java 8+ converts to Red-Black Tree if bucket size > 8. Worst case without treeify: $O(n)$. With treeify: $O(\log n)$.

3.2 LinkedHashMap — Internal Implementation

LinkedHashMap extends **HashMap** and adds a doubly-linked list across all entries. It maintains insertion order (or access order if configured).

LinkedHashMap extends HashMap — adds prev/next pointers to each Node:

```

class LinkedHashMap.Entry<K,V> extends HashMap.Node<K,V> {
    Entry<K,V> before, after; // doubly linked list pointers
}

```

Hash table (for $O(1)$ access):	Linked list (for order):
[0] -> null	head <--> [Alice,95] <--> [Bob,88] <-
-> [Eve,72] <--> tail	
[1] -> [Alice,95]	
[5] -> [Bob,88] -> [Eve,72]	Iteration follows linked list ->
Alice, Bob, Eve	

Access-order mode (for LRU Cache):
 After get("Bob") -> Bob moves to tail: Alice <-> Eve <-> Bob

- Maintains INSERTION order by default
- Can be configured for ACCESS order: `new LinkedHashMap<>(16, 0.75f, true)`
- Access order: most-recently accessed element moves to tail — perfect for LRU Cache
- Slightly more memory than HashMap — extra two pointers per entry

```
// LRU Cache using LinkedHashMap:
class LRUCache<K,V> extends LinkedHashMap<K,V> {
    private int capacity;
    LRUCache(int cap) {
        super(cap, 0.75f, true); // accessOrder = true
        this.capacity = cap;
    }
    @Override
    protected boolean removeEldestEntry(Map.Entry<K,V> eldest) {
        return size() > capacity; // remove least-recently-used
    }
}
```

3.3 TreeMap — Internal Implementation

TreeMap uses a Red-Black Tree where keys are always maintained in sorted order. Implements **NavigableMap**.

TreeMap — Red-Black Tree, keys sorted:

```
put('B',2), put('A',1), put('D',4), put('C',3), put('E',5)
```

```

      'C' (BLACK)                                <- root (automatically balanced)
     /      \
  'B' (BLACK)  'D' (BLACK)
   /      \
'A' (RED)   'E' (RED)
```

Iteration (in-order traversal) = A, B, C, D, E (always sorted)

Keys must implement Comparable OR provide Comparator
 NULL keys -> throws NullPointerException (can't call compareTo(null))

- $O(\log n)$ for get, put, remove, containsKey
- Does NOT allow null keys — `compareTo(null)` throws `NullPointerException`
- Null values ARE allowed
- Implements `NavigableMap`: `floorKey()`, `ceilingKey()`, `lowerKey()`, `higherKey()`

```
TreeMap<String, Integer> tm = new TreeMap<>();
```

```

tm.put("Banana", 2);
tm.put("Apple", 1);
tm.put("Mango", 3);
// Sorted: {Apple=1, Banana=2, Mango=3}

tm.firstKey();           // "Apple"
tm.lastKey();            // "Mango"
tm.floorKey("Cherry");   // "Banana" (largest key <= Cherry)
tm.ceilingKey("Cherry"); // "Mango" (smallest key >= Cherry)
tm.headMap("Mango");     // {Apple=1, Banana=2} (keys < Mango)
tm.tailMap("Banana");    // {Banana=2, Mango=3} (keys >= Banana)
tm.subMap("Apple", "Mango"); // {Apple=1, Banana=2}

```

3.4 Hashtable — Legacy Class

- Legacy class from Java 1.0 — synchronized on every method
- Does NOT allow null keys or null values — throws NullPointerException
- No order maintained
- Slower than HashMap due to synchronization overhead on every call
- Modern replacement: ConcurrentHashMap (segment-based locking, much faster)

Feature	HashMap	LinkedHashMap	TreeMap	Hashtable
Order	No order	Insertion order	Sorted by key	No order
Null key	1 allowed	1 allowed	NOT allowed	NOT allowed
Null values	Allowed	Allowed	Allowed	NOT allowed
Thread-safe	No	No	No	Yes (legacy)
Performance	O(1) avg	O(1) avg	O(log n)	O(1) slower
Internal struct	Array+LinkedList/Tree	Array+LinkedList+List	Red-Black Tree	Array+LinkedList
Introduced	Java 1.2	Java 1.4	Java 1.2	Java 1.0

4. QUEUE INTERFACE

Core Characteristics

- Follows FIFO (First In, First Out) by default
- Elements added at REAR (tail) and removed from FRONT (head)
- Extends Collection interface
- Allows duplicate elements
- PriorityQueue does NOT follow FIFO — retrieves in priority order
- Deque (Double Ended Queue) allows insert/remove from both ends

QUEUE — FIFO (First In, First Out):

```
enqueue -> [ 'D' | 'C' | 'B' | 'A' ] -> dequeue
(add at rear)  REAR                FRONT (remove from front)

add('A') -> ['A']
add('B') -> ['A', 'B']
add('C') -> ['A', 'B', 'C']
poll()    -> removes 'A' -> ['B', 'C']    (A was first in)
peek()    -> returns 'B' (next to be removed, not removed yet)
```

Queue Method Pairs — Must Know!

Operation	Throws Exception (if fail)	Returns null/false (if fail)	Which to prefer
Insert	add(e)	offer(e)	offer() — safer in bounded queues
Remove head	remove()	poll()	poll() — returns null if empty
Examine head	element()	peek()	peek() — returns null if empty

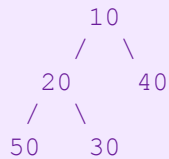
★ **Interview Q:** Difference between add() and offer()? In unbounded queues (like LinkedList), both behave same. In bounded queues, add() throws IllegalStateException when full; offer() returns false. Always prefer offer(), poll(), peek() — they don't throw exceptions.

4.1 PriorityQueue — Internal Implementation

PriorityQueue is backed by a **min-heap** (binary heap stored in an Object[] array). The head is always the SMALLEST element.

```
PriorityQueue<Integer> pq with elements: 50, 20, 40, 10, 30
```

Min-Heap tree (logical):



Heap properties:

- Parent $\leq$ children (min-heap)
- Complete binary tree

Physical array (how it's stored):

index: 0 1 2 3 4
array: [10, 20, 40, 50, 30]

For node at index i :

left child = $2i + 1$

right child = $2i + 2$

parent = $(i-1) / 2$

poll() removes index 0 (smallest=10):

-> move last element (30) to root -> heapify down -> $O(\log n)$

result: [20, 30, 40, 50] with 20 at head

- Default: min-heap — smallest element at head
- Does NOT maintain insertion order — elements are reordered by heap property
- Allows duplicate elements
- Does NOT allow null elements
- $O(\log n)$ for add() and poll()
- $O(1)$ for peek() — just return array[0]
- $O(n)$ for contains() and remove(Object) — linear scan
- NOT thread-safe — use PriorityQueue for thread safety

```
// Min-heap (default):
PriorityQueue<Integer> minPQ = new PriorityQueue<>();
minPQ.add(50); minPQ.add(20); minPQ.add(10); minPQ.add(30);
minPQ.poll(); // 10 (smallest always at head)
minPQ.poll(); // 20
minPQ.peek(); // 30 (next smallest, not removed)

// Max-heap using Comparator.reverseOrder():
PriorityQueue<Integer> maxPQ = new PriorityQueue<>(Comparator.reverseOrder());
maxPQ.add(50); maxPQ.add(20); maxPQ.add(10);
maxPQ.poll(); // 50 (largest at head)

// Custom object priority:
PriorityQueue<int[]> tasks = new PriorityQueue<>((a,b) -> a[1] - b[1]);
tasks.add(new int[]{1, 3}); // task 1, priority 3
tasks.add(new int[]{2, 1}); // task 2, priority 1 <- higher priority
tasks.poll(); // [2, 1] lowest priority number = highest priority
```

4.2 ArrayDeque — Internal Implementation

ArrayDeque implements **Deque** using a circular array. Elements can be added/removed from BOTH ends efficiently.

ArrayDeque — Circular Array (resizable):

```
elements[] = [ null | 'C' | 'D' | 'E' | null | 'A' | 'B' ]
```

```

indices:      0      1      2      3      4      5      6
              ^head                    ^tail
head=1, tail=6 (circular – wraps around the array)

addFirst('Z'): tail moves left -> elements[0] = 'Z', tail=0
addLast('F'):  head moves right -> elements[4] = 'F', head=4

Why circular? Avoids shifting elements when adding at front!
Both addFirst() and addLast() are O(1) amortized

```

- Default initial capacity: 16, grows by doubling
- Does NOT allow null elements
- NOT thread-safe
- Faster than LinkedList for most Deque operations (no node allocation overhead)
- Preferred over Stack (legacy) and LinkedList for stack/queue

```

ArrayDeque<String> deque = new ArrayDeque<>();

// As a Queue (FIFO):
deque.offer("A");    // add at tail
deque.offer("B");
deque.poll();        // remove from head -> 'A'

// As a Stack (LIFO):
deque.push("X");     // add at head (same as addFirst)
deque.push("Y");     // add at head -> Y is now on top
deque.pop();         // remove from head -> 'Y'

// Deque operations:
deque.addFirst("F"); // add to front
deque.addLast("L");  // add to back
deque.peekFirst();   // look at front without removing
deque.peekLast();    // look at back without removing
deque.pollFirst();   // remove from front
deque.pollLast();    // remove from back

```

Feature	Stack	ArrayDeque	LinkedList (as Queue)
Thread-safe	Yes (synchronized)	No	No
Null elements	Yes (extends Vector)	NO	YES
Performance	Slower (sync)	Fastest	Medium (node alloc)
Memory	Medium	Less (array)	More (node pointers)
Double-ended	No	YES	YES
Recommended	Legacy only	YES — modern choice	When null needed

5. ITERATORS — fail-fast vs fail-safe

What is an Iterator?

- Iterator is a design pattern to traverse a collection without exposing its structure
- `Iterator<E>` interface: `hasNext()`, `next()`, `remove()`
- `ListIterator<E>` (List only): adds `hasPrevious()`, `previous()`, `add()`, `set()`

fail-fast Iterators

fail-fast iterators throw `ConcurrentModificationException` if the collection is modified during iteration (other than through the iterator itself).

fail-fast mechanism:

Collection maintains a `modCount` (modification count)
 Iterator captures `expectedModCount = modCount` at creation

On every `next()` call:
 if (`modCount != expectedModCount`) throw new
`ConcurrentModificationException()`

Collections using fail-fast: `ArrayList`, `LinkedList`, `HashMap`, `HashSet`,
`TreeMap`, `TreeSet`, `LinkedHashMap`,
`LinkedHashSet`

```
List<String> list = new ArrayList<>(Arrays.asList("A", "B", "C"));
Iterator<String> it = list.iterator();
while (it.hasNext()) {
    String s = it.next();
    if (s.equals("B")) {
        list.remove(s);    // WRONG! throws ConcurrentModificationException
        it.remove();       // CORRECT! use iterator's own remove()
    }
}
```

fail-safe Iterators

fail-safe iterators work on a COPY of the collection. Modifications don't affect the in-progress iteration. No `ConcurrentModificationException`.

fail-safe mechanism (`CopyOnWriteArrayList` example):

Original list: `[A, B, C]`
 Iterator created -> works on snapshot copy: `[A, B, C]`
`list.add('D')` -> creates a NEW copy: `[A, B, C, D]`
 Iterator continues on OLD copy `[A, B, C]` -> no exception

Downside: snapshot may be stale (doesn't see new additions)

More memory: copy created on every write

Collections using fail-safe: CopyOnWriteArrayList, ConcurrentHashMap

Property	fail-fast	fail-safe
Exception	ConcurrentModificationException	No exception
Works on	Original collection	Copy/snapshot
Modification during iteration	NOT allowed (throws)	Allowed (sees old data)
Memory	No extra copy	Extra copy created
Examples	ArrayList, HashMap, HashSet	CopyOnWriteArrayList, ConcurrentHashMap

6. Comparable vs Comparator

Comparable — Natural Ordering

- Interface: `java.lang.Comparable<T>`
- Method: `int compareTo(T other)`
- Implemented inside the class itself — defines natural order
- Used by: `TreeSet`, `TreeMap`, `Collections.sort()` by default

```
class Student implements Comparable<Student> {
    String name;
    int marks;

    @Override
    public int compareTo(Student other) {
        return this.marks - other.marks; // sort by marks ascending
        // return this.name.compareTo(other.name); // or by name
    }
}

List<Student> students = new ArrayList<>();
Collections.sort(students); // uses compareTo() automatically
```

Comparator — Custom Ordering

- Interface: `java.util.Comparator<T>`
- Method: `int compare(T o1, T o2)`
- Defined OUTSIDE the class — multiple sort orders possible
- Used when: you don't own the class OR need multiple orderings

```
// Sort by name:
Comparator<Student> byName = (a, b) -> a.name.compareTo(b.name);

// Sort by marks descending:
Comparator<Student> byMarksDesc = (a, b) -> b.marks - a.marks;

// Chaining comparators:
Comparator<Student> byNameThenMarks =
    Comparator.comparing((Student s) -> s.name)
                .thenComparingInt(s -> s.marks);

Collections.sort(students, byName);
students.sort(byMarksDesc);

// TreeSet with custom Comparator:
TreeSet<Student> ts = new TreeSet<>(byName);
```


Property	Comparable	Comparator
Package	java.lang	java.util
Method	compareTo(T o)	compare(T o1, T o2)
Defined in	Inside the class	Outside the class (separate class/lambda)
Number of orderings	Only one (natural)	Multiple possible
Modifies class	Yes	No
Best used for	Natural/default order	Ad-hoc/multiple orderings

7. java.util.Collections — Utility Methods

Most Important Methods

Method	Description	Time Complexity
sort(list)	Sort list in natural order	$O(n \log n)$
sort(list, comparator)	Sort list with custom order	$O(n \log n)$
binarySearch(list, key)	Binary search (list must be sorted)	$O(\log n)$
reverse(list)	Reverse order of elements	$O(n)$
shuffle(list)	Randomly shuffle elements	$O(n)$
min(collection)	Smallest element (natural order)	$O(n)$
max(collection)	Largest element (natural order)	$O(n)$
frequency(collection, obj)	Count occurrences of obj	$O(n)$
fill(list, obj)	Replace all elements with obj	$O(n)$
copy(dest, src)	Copy src into dest	$O(n)$
nCopies(n, obj)	Immutable list with n copies of obj	$O(1)$
disjoint(c1, c2)	True if no common elements	$O(n)$
unmodifiableList(list)	Returns read-only view	$O(1)$
synchronizedList(list)	Returns thread-safe view	$O(1)$
singletonList(obj)	Immutable list with 1 element	$O(1)$
emptyList()	Immutable empty list	$O(1)$
swap(list, i, j)	Swap elements at indices i and j	$O(1)$
rotate(list, distance)	Rotate elements by distance	$O(n)$

```

List<Integer> nums = new ArrayList<>(Arrays.asList(3,1,4,1,5,9,2,6));

Collections.sort(nums);           // [1,1,2,3,4,5,6,9]
Collections.reverse(nums);        // [9,6,5,4,3,2,1,1]
Collections.shuffle(nums);         // random order
Collections.binarySearch(nums, 5); // works only if sorted first!
Collections.max(nums);             // 9
Collections.min(nums);            // 1
Collections.frequency(nums, 1);    // 2

List<Integer> readOnly = Collections.unmodifiableList(nums);
readOnly.add(10); // throws UnsupportedOperationException

List<Integer> syncList = Collections.synchronizedList(new ArrayList<>());

```

8. Thread Safety in Collections

Which collections are thread-safe?

Collection	Thread-Safe?	How?	Modern Alternative
ArrayList	NO	Not synchronized	CopyOnWriteArrayList
LinkedList	NO	Not synchronized	ConcurrentLinkedQueue
HashMap	NO	Not synchronized	ConcurrentHashMap
HashSet	NO	Not synchronized	ConcurrentHashMap.newKeySet()
Vector	YES	synchronized methods (legacy)	Use ArrayList instead
Hashtable	YES	synchronized methods (legacy)	ConcurrentHashMap
Stack	YES	Extends Vector (legacy)	ArrayDeque
ConcurrentHashMap	YES	Segment/bucket-level locking	Preferred for maps
CopyOnWriteArrayList	YES	Copy array on every write	Read-heavy scenarios
Collections.synchronizedX()	YES	Wraps every method in sync block	Quick fix only

ConcurrentHashMap vs Hashtable

Hashtable: synchronized on ENTIRE map (one lock for all)
 Thread 1 -> lock entire table -> put(k1,v1) -> unlock
 Thread 2 -> BLOCKED until Thread 1 finishes -> very slow

ConcurrentHashMap (Java 8+): bucket-level locking
 Thread 1 -> lock bucket[5] -> put(k1,v1) -> unlock bucket[5]
 Thread 2 -> lock bucket[9] -> put(k2,v2) -> unlock bucket[9]
 Both threads can work SIMULTANEOUSLY on different buckets!

Result: ConcurrentHashMap is much faster than Hashtable
 In Java 8: uses CAS (Compare-And-Swap) operations for even better perf

9. Master Comparison — All Collections

Class	Interface	Order	Duplicates	Null	Thread-Safe	Time Complexity
ArrayList	List	Insertion	YES	YES	NO	get O(1), add/rem O(n)
LinkedList	List+Deque	Insertion	YES	YES	NO	get O(n), add/rem O(1) ends
Vector	List	Insertion	YES	YES	YES	Same as ArrayList
Stack	List	LIFO	YES	YES	YES	push/pop O(1)
HashSet	Set	No order	NO	1 null	NO	O(1) avg
LinkedHashSet	Set	Insertion	NO	1 null	NO	O(1) avg
TreeSet	Set	Sorted	NO	NO	NO	O(log n)
HashMap	Map	No order	Keys NO	1 null key	NO	O(1) avg
LinkedHashMap	Map	Insertion	Keys NO	1 null key	NO	O(1) avg
TreeMap	Map	Sorted key	Keys NO	NO keys	NO	O(log n)
Hashtable	Map	No order	Keys NO	NO	YES	O(1) slower
PriorityQueue	Queue	Priority	YES	NO	NO	add/poll O(log n)
ArrayDeque	Deque	LIFO/FIFO	YES	NO	NO	O(1) both ends
ConcurrentHashMap	Map	No order	Keys NO	NO	YES	O(1) avg
CopyOnWriteArrayList	List	Insertion	YES	YES	YES	read O(1), write O(n)

10. When to Use What?

Scenario / Requirement	Best Choice	Why
Fast index-based access, read-heavy	ArrayList	O(1) random access
Frequent insert/delete at head or tail	LinkedList	O(1) at both ends
Thread-safe list, frequent reads	CopyOnWriteArrayList	No lock for reads
Thread-safe list, frequent writes	Collections.synchronizedList(ArrayList)	Simpler for write-heavy
LIFO stack in modern Java	ArrayDeque	Faster than Stack, not legacy
Unique elements, no order needed	HashSet	Fastest Set, O(1)
Unique elements + insertion order	LinkedHashSet	Order preserved
Unique elements + sorted order	TreeSet	Always sorted, O(log n)
Key-value, no order, fastest	HashMap	O(1) average
Key-value + insertion order	LinkedHashMap	Maintains order
Key-value + sorted by key	TreeMap	O(log n), NavigableMap
Key-value, thread-safe (modern)	ConcurrentHashMap	Segment locking, fast
Key-value, thread-safe (legacy)	Hashtable	Legacy only — avoid
Process by priority	PriorityQueue	Min-heap, O(log n) poll
FIFO queue, no priority	LinkedList / ArrayDeque	Both work; ArrayDeque faster
LRU Cache implementation	LinkedHashMap (accessOrder=true)	Built-in eviction support
Counting word frequencies	HashMap<String,Integer>	Key=word, value=count
Finding first non-repeated char	LinkedHashMap	Order + uniqueness
Graph adjacency list	HashMap<Integer,List<Integer>>	Map of node to neighbors

11. Top 30 Interview Questions & Answers

Q1 Is Map part of the Collection interface?

► No. Map is part of the Collections Framework but does NOT extend the Collection interface. It has its own separate hierarchy. You cannot use enhanced for-loop directly on a Map.

Q2 How does HashSet prevent duplicates?

► When add(e) is called: (1) compute hashCode(e) to find bucket index, (2) if bucket has entries, call equals() on each. If equals() returns true -> duplicate found, NOT inserted. If false -> insert. This is why you must override BOTH hashCode() and equals() for custom objects.

Q3 What happens if hashCode() always returns the same value?

► All elements go into the same bucket — degenerating the HashMap into a linked list. O(1) operations become O(n). This is called a hash collision attack. Java 8 mitigates this by treeifying buckets with >8 nodes (O(log n)).

Q4 ArrayList vs LinkedList — when to use which?

► ArrayList: use when you need frequent random access (get by index) or mostly add at end. LinkedList: use when you need frequent insertions/deletions at the beginning or end and do NOT need random access.

Q5 What is the difference between HashMap and Hashtable?

► HashMap: not synchronized, allows 1 null key and multiple null values, Java 1.2. Hashtable: synchronized (thread-safe), does NOT allow null keys or values, legacy class (Java 1.0). Use ConcurrentHashMap instead of Hashtable in modern code.

Q6 What is the load factor in HashMap? Why is it 0.75?

► Load factor determines when HashMap is resized. Default is 0.75 — rehashing occurs when 75% of capacity is filled. 0.75 is a balance: higher value = more collisions (slower); lower value = more memory waste. When size > capacity * 0.75, capacity doubles and all entries are rehashed.

Q7 What is fail-fast vs fail-safe iterator?

► fail-fast: throws ConcurrentModificationException if collection is modified during iteration (tracked via modCount). Examples: ArrayList, HashMap. fail-safe: works on a copy, no exception but may see stale data. Examples: CopyOnWriteArrayList, ConcurrentHashMap.

Q8 Difference between Comparable and Comparator?

► Comparable: implemented inside the class, defines natural/default ordering, one sort sequence. Comparator: external class or lambda, defines custom ordering, multiple sort

sequences possible. TreeSet/TreeMap use Comparable by default; accept Comparator in constructor.

Q9 What is a ConcurrentModificationException?

➤ Thrown by fail-fast iterators when a collection is structurally modified after the iterator is created (other than through the iterator's own remove()). Fix: use iterator.remove(), or CopyOnWriteArrayList, or iterate over a copy.

Q10 What is the contract between hashCode() and equals()?

➤ (1) If a.equals(b) is true, then a.hashCode() MUST == b.hashCode(). (2) If a.hashCode() == b.hashCode(), a.equals(b) may or may not be true (collision). Violating rule 1 breaks HashMap, HashSet, Hashtable.

Q11 Why does TreeMap not allow null keys?

➤ TreeMap uses compareTo() (or Comparator.compare()) to order keys. Calling compareTo(null) throws NullPointerException. TreeMap values (not keys) can be null. HashMap allows one null key because it handles null key with a special case (always at bucket[0]).

Q12 What is the internal implementation of TreeSet?

➤ TreeSet is backed by a TreeMap. Elements are stored as keys of the TreeMap with a dummy PRESENT object as value. TreeMap uses a Red-Black Tree (self-balancing BST) ensuring $O(\log n)$ for add, remove, contains and always sorted order.

Q13 What is the difference between add() and offer() in Queue?

➤ Both insert an element. add() throws IllegalStateException if the queue is full (bounded queues). offer() returns false if the queue is full. For most implementations (LinkedList, PriorityQueue), they behave the same since they're unbounded. Prefer offer() as it's safer.

Q14 What is PriorityQueue internally? How does it order elements?

➤ PriorityQueue is backed by a min-heap (Object[] array). Elements are arranged so the smallest is always at index 0. For a node at index i : left child = $2i+1$, right child = $2i+2$, parent = $(i-1)/2$. add() does sift-up, poll() removes root and does sift-down. $O(\log n)$ for both.

Q15 Difference between Queue and Deque?

➤ Queue: single-ended — insert at rear, remove from front (FIFO). Deque (Double-Ended Queue): insert and remove from BOTH ends. ArrayDeque implements Deque and can serve as both a Queue (offer/poll) and a Stack (push/pop).

Q16 What is CopyOnWriteArrayList? When to use?

➤ Thread-safe ArrayList where every write operation (add, set, remove) creates a fresh copy of the underlying array. Reads are lock-free and very fast. Best for read-heavy, write-rarely

scenarios (e.g., observer/listener lists). Expensive writes — not suitable if frequent modifications.

Q17 How does LinkedHashMap maintain insertion order?

► LinkedHashMap extends HashMap and adds a doubly-linked list running through all its entries. Each Entry has before and after pointers. On insertion, the new entry is appended to the tail of this list. On iteration, the list is followed — giving insertion order.

Q18 What is the difference between HashSet and TreeSet?

► HashSet: backed by HashMap, no order, $O(1)$, allows null. TreeSet: backed by TreeMap (Red-Black Tree), sorted order, $O(\log n)$, no null keys. Use HashSet for fastest add/contains; use TreeSet when sorted traversal is needed.

Q19 Can we sort a HashSet?

► HashSet itself cannot be sorted (no index). To sort: (1) convert to ArrayList: `new ArrayList<>(set)`, then `Collections.sort()`. (2) Or use TreeSet: `new TreeSet<>(set)` — automatically sorted. (3) Or stream: `set.stream().sorted().collect(Collectors.toList())`.

Q20 What is the difference between Iterator and ListIterator?

► Iterator: works on any Collection, forward-only, supports `remove()`. ListIterator: List only, bidirectional (has `Previous/previous`), supports `add()`, `set()`, `nextIndex()`, `previousIndex()`. ListIterator allows modification during iteration.

Q21 What is Collections.unmodifiableList()?

► Returns a read-only VIEW of the original list. Any attempt to modify it (add, remove, set) throws `UnsupportedOperationException`. It is NOT a copy — changes to the original list are reflected in the unmodifiable view. For a truly independent immutable list, use `List.copyOf()` (Java 10+).

Q22 How to make a thread-safe ArrayList?

► Three options: (1) `Collections.synchronizedList(new ArrayList<>())` — wraps every method, manual sync needed during iteration. (2) `CopyOnWriteArrayList` — best for read-heavy. (3) Use explicit locks (`ReentrantLock`) if fine-grained control needed.

Q23 What is the difference between Collection and Collections?

► Collection (interface): root interface for List, Set, Queue. Collections (utility class): `java.util.Collections` — provides static helper methods: `sort()`, `binarySearch()`, `reverse()`, `shuffle()`, `synchronizedList()`, `unmodifiableList()`, etc.

Q24 What is a WeakHashMap?

► A HashMap where keys are stored as WeakReferences. If a key has no other strong references, it can be garbage collected — and the entry is automatically removed. Used for caches where entries should expire when the key is no longer in use.

Q25 What is EnumSet and EnumMap?

► EnumSet: a Set implementation for use with enum types. Internally uses a bit vector — extremely compact and fast. EnumMap: a Map with enum keys. Both are specialized for enum keys and faster than HashSet/HashMap for enum operations.

Q26 What is the difference between peek() and element() in Queue?

► Both return the head element without removing it. peek() returns null if the queue is empty. element() throws NoSuchElementException if the queue is empty. Always prefer peek() to avoid exceptions.

Q27 How does HashMap handle rehashing?

► When size > capacity * loadFactor: (1) Create new array of double the capacity. (2) For each existing entry: recompute bucket index = hash & (newCapacity - 1). (3) Insert into new array. This is O(n). In Java 8+, entries in treeified buckets are split efficiently.

Q28 Can a HashMap have two keys with same hashCode?

► Yes — this is a collision. Both entries coexist in the same bucket (linked list or Red-Black Tree). equals() is used to distinguish them. If hashCode is same AND equals() returns true, it's a duplicate key — value is overwritten.

Q29 What is the complexity of HashMap.get() in worst case?

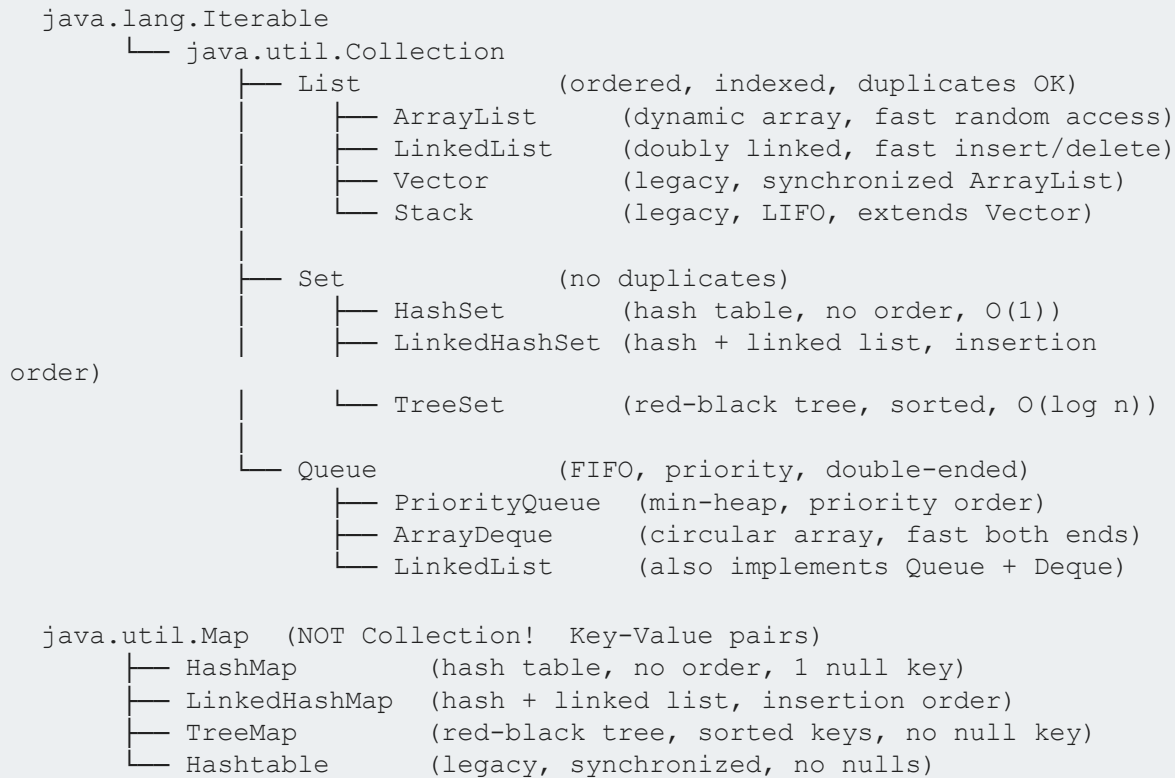
► O(n) before Java 8 — if all keys hash to same bucket (linked list traversal). O(log n) in Java 8+ — because buckets with >8 nodes are treeified into Red-Black Trees. Average case is always O(1) with a good hashCode.

Q30 Explain the internal flow of HashMap.put(key, value).

► (1) Null key? -> special handling, bucket 0. (2) hash = hashCode(key) ^ (hashCode >> 16) [spreads bits]. (3) index = (n-1) & hash. (4) Bucket empty? -> insert new Node. (5) Bucket has nodes? -> check each: same hash + equals true? -> update value. No match -> add node at tail. (6) If size > threshold -> resize (double capacity, rehash).

12. Final One-Page Cheat Sheet

Collections Framework at a Glance



Key Rules to Never Forget

- **Map is NOT a Collection — separate interface hierarchy**
- **TreeSet / TreeMap — NO null keys (compareTo throws NPE)**
- **HashSet is backed by HashMap; TreeSet is backed by TreeMap**
- **Override BOTH hashCode() AND equals() for custom objects in Hash-based collections**
- **fail-fast = ConcurrentModificationException; fail-safe = works on copy**
- **ArrayList grows 1.5x; Vector grows 2x (doubles)**
- **Prefer ArrayDeque over Stack; prefer ConcurrentHashMap over Hashtable**
- **PriorityQueue is a min-heap — smallest element always at head**
- **LinkedHashMap with accessOrder=true -> LRU Cache**
- **HashMap default capacity=16, load factor=0.75, resizes when size > 12**
- **Java 8+: HashMap treeifies buckets with >8 nodes -> O(log n) worst case**

□ **Final Tip:** In every interview answer about collections, always mention: the internal data structure (array/linked list/red-black tree), time complexities, null handling, thread safety, and which class to prefer in modern Java. That covers 95% of collection questions!

